Analisi e confronto dei metodi Velocity-Verlet e Runge-Kutta 4 per il pendolo semplice

Autore:

Alberto Demarco

Sotto la guida di

Prof. Giuseppe Magnifico

Dipartimento Interuniversitario di Fisica "Michelangelo Merlin"

Università degli Studi di Bari "Aldo Moro"

A.A. 2025/2026

Indice

1 Introduzione **1**

2 Grafici e Risultati **1**

3 Discussione dei Risultati **2**
3.1 Traiettorie e Spazio delle fasi 2
3.2 Conservazione dell'Energia 2
3.3 Ordine di Convergenza e np.polyfit 3

4 Conclusioni **3**

5 Appendice: Codice Python **4**

Abstract

L'obiettivo dell'esperienza è l'implementazione in Python e lo studio dei metodi numerici Velocity-Verlet e Runge-Kutta del 4° ordine (RK4), applicati all'integrazione delle equazioni del moto per il pendolo semplice. Vengono confrontati l'accuratezza delle traiettorie, la capacità di conservazione dell'energia meccanica a lungo termine e l'ordine di convergenza dei due algoritmi.

1 Introduzione

Le equazioni del moto per il pendolo semplice possono essere scritte introducendo la velocità angolare come un sistema di due equazioni differenziali ordinarie (ODE) del primo ordine:

$$\frac{d\theta}{dt} = \omega \tag{1}$$

$$\frac{d\omega}{dt} = -\frac{g}{L}\sin(\theta) \tag{2}$$

Per risolvere numericamente questo sistema, analizziamo due metodi distinti.

Metodo Velocity-Verlet: È un integratore numerico del secondo ordine con errore globale $O(h^2)$, ampiamente utilizzato per la sua stabilità. Le sue tre equazioni di aggiornamento per passare dal passo n al passo $n+1$ sono:

$$\theta_{n+1} = \theta_n + h\omega_n + \frac{1}{2}h^2 a_n \tag{3}$$

$$a_{n+1} = -\frac{g}{L}\sin(\theta_{n+1}) \tag{4}$$

$$\omega_{n+1} = \omega_n + \frac{1}{2}h(a_n + a_{n+1}) \tag{5}$$

Essendo un metodo simplettico, la sua principale proprietà è quella di far oscillare l'energia meccanica attorno al valore corretto senza che questa subisca derive sistematiche nel tempo.

Metodo Runge-Kutta del 4° Ordine (RK4): È uno dei metodi più diffusi per le ODE, basato sul calcolo di quattro pendenze intermedie (k_1, k_2, k_3, k_4) che vengono successivamente composte in una media pesata. Pur garantendo un errore globale molto basso proporzionale a $O(h^4)$, questo metodo non è simplettico e richiede quattro valutazioni della forza per ogni passo. Tale assenza di simpletticità si traduce nel rischio di una lenta deriva energetica in caso di simulazioni molto estese.

2 Grafici e Risultati

In questa sezione si riportano i risultati grafici della simulazione in Python, con parametri $L = 1$ m e $\theta_0 = 0.5$ rad.

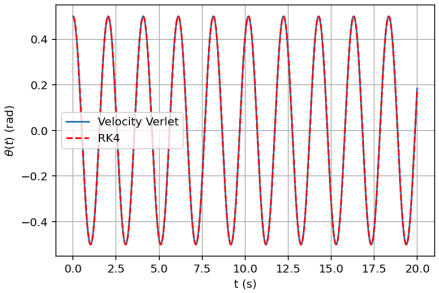

Figura 1: Esercizio 1: Andamento nel tempo $\theta(t)$ sovrapposto per i due algoritmi ($h = 0.01$ s, $t_{max} = 20$ s).

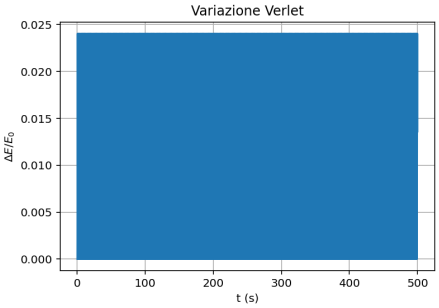

Figura 2: Esercizio 2: Variazione percentuale dell'energia $\Delta E/E_0$ nel tempo per Velocity-Verlet.

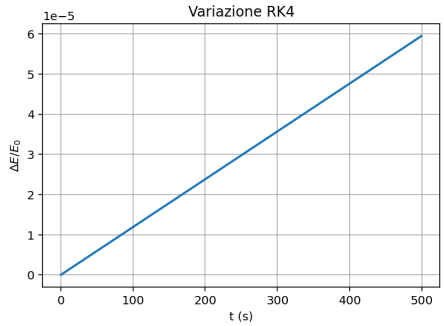

Figura 3: Esercizio 2: Variazione percentuale dell'energia $\Delta E/E_0$ per il metodo RK4.

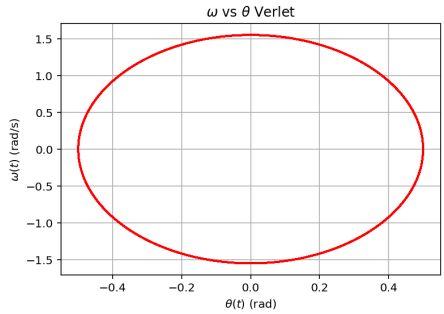

Figura 4: Esercizio 3: Spazio delle fasi (ω vs θ) simulato tramite Velocity-Verlet.

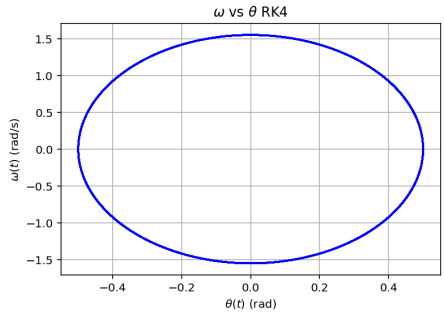

Figura 5: Esercizio 3: Spazio delle fasi (ω vs θ) simulato tramite RK4.

Metodo	Ord. Misurato	Atteso
Velocity-Verlet	1.99	2
RK4	4.04	4

Tabella 1: Stima dell'ordine di convergenza.

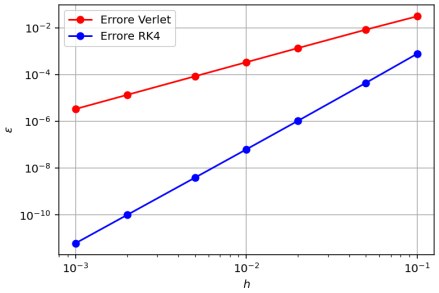

Figura 6: Esercizio 3: Errore globale a $t = 10$ s al variare del passo di integrazione h, visualizzato su scala logaritmica.

3 Discussione dei Risultati

3.1 Traiettorie e Spazio delle fasi

L'Esercizio 1 mostra come per un tempo di 20 s e un passo moderato ($h = 0.01$ s), le traiettorie generate da Velocity-Verlet e RK4 siano essenzialmente indistinguibili a occhio nudo (Figura 1). Lo spazio delle fasi, simmetrico e periodico in Figura 4 e 5, corrobora il fatto che per tempi discretamente brevi e ampiezze iniziali ristrette i due metodi catturino in egual misura la fisica di base dell'oscillatore pendolare.

3.2 Conservazione dell'Energia

Il quadro cambia per tempi lunghi (Esercizio 2, $t_{max} = 500$ s). Come deducibile dalle curve in Figura 2 e Figura 3, Velocity-Verlet conferma la sua natura simplettica mantenendo le oscillazioni dell'errore relativo di energia controllate entro un certo limite. Il metodo RK4, essendo privo di tale proprietà intrinseca, comincia a perdere progressivamente stabilità sull'energia a causa dell'accumulo di errori non compensabili.

3.3 Ordine di Convergenza e np.polyfit

Nell'ultimo studio (Esercizio 3) è stato ricavato graficamente l'ordine di convergenza dei due algoritmi. In campo logaritmico, la legge di dipendenza dell'errore globale $\epsilon \propto h^p$ si linearizza in $\log(\epsilon) \approx p \log(h) + C$. Per determinare la pendenza della retta p nel piano log-log (Figura 6) è stata utilizzata la funzione `np.polyfit(log_h, np.log(errore), 1)` di Python, che agisce calcolando l'interpolazione ai minimi quadrati.

I risultati misurati e stampati in Tabella 1 riflettono fedelmente il modello teorico: `polyfit` restituisce coefficienti pari a 1.99 per il Velocity-Verlet (ordine atteso $O(h^2)$) e 4.04 per RK4 (ordine atteso $O(h^4)$).

4 Conclusioni

La scelta del metodo ideale dipende fortemente dai fini della simulazione. Qualora ci si trovi a voler riprodurre orbite e moti planetari prolungati nel tempo, dove l'esatta conservazione dei vincoli fisici e meccanici sul lungo periodo è fondamentale, Velocity-Verlet è di gran lunga lo strumento preferibile. Se invece lo scopo è ottenere estrema fedeltà analitica sulle posizioni e i parametri cinematici nel corso di una dinamica circoscritta e rapida, l'elevato ordine di RK4 risulta l'opzione migliore, compensando lo svantaggio computazionale delle 4 chiamate per iterazione.

5 Appendice: Codice Python

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 23 09:20:25 2026

@author: alberto
"""

import numpy as np
from matplotlib import pyplot as plt

#Esercizio 1

def velocity_verlet(t, h, theta_0, omega_in, a_0, L, g):

    theta = np.zeros(len(t))
    omega = np.zeros(len(t))
    a     = np.zeros(len(t))

    theta[0] = theta_0
    omega[0] = omega_in
    a[0]     = a_0

    for i in range(len(t) - 1):

        theta[i+1] = theta[i] + h*omega[i] + 0.5*(h**2)*a[i]
        a[i+1]     = -(g/L)*np.sin(theta[i+1])
        omega[i+1] = omega[i] + 0.5*h*(a[i]+a[i+1])

    return theta, omega, a

def RK4(t, h, theta_0, omega_in, L, g):

    theta = np.zeros(len(t))
    omega = np.zeros(len(t))

    theta[0] = theta_0
    omega[0] = omega_in

    for i in range(len(t) - 1):

        k_1_theta = omega[i]
        k_1_omega = -(g/L)*np.sin(theta[i])

        k_2_theta = omega[i] + (h/2)*k_1_omega
        k_2_omega = -(g/L)*np.sin(theta[i] + (h/2)*k_1_theta)

        k_3_theta = omega[i] + (h/2)*k_2_omega
        k_3_omega = -(g/L)*np.sin(theta[i] + (h/2)*k_2_theta)

        k_4_theta = omega[i] + h*k_3_omega
        k_4_omega = -(g/L)*np.sin(theta[i] + h*k_3_theta)

        theta[i+1] = theta[i] + (h/6)*(k_1_theta + 2*k_2_theta + 2*k_3_theta + k_4_theta)
        omega[i+1] = omega[i] + (h/6)*(k_1_omega + 2*k_2_omega + 2*k_3_omega + k_4_omega)

    return theta, omega

```

```python
h = 0.01 #s
t_max = 20.0 #s
L = 1 #m
theta_0 = 0.5 #rad
g = 9.81 #m/s^2
t_0 = 0.0 #s
omega_in = 0.0 #rad/s
a_0 = -(g/L)*np.sin(theta_0)

t = np.arange(t_0, t_max, h)

theta_verlet, omega_verlet, a_verlet = velocity_verlet(t, h, theta_0, omega_in, a_0, L, g)
theta_RK4, omega_RK4 = RK4(t, h, theta_0, omega_in, L, g)

plt.plot(t, theta_verlet, label="Velocity Verlet")
plt.plot(t, theta_RK4, "r--", label="RK4")
plt.xlabel("t (s)")
plt.ylabel(r"$\theta (t)$ (rad)")
plt.legend()
plt.grid(True)
plt.show()

#Esercizio 2

h = 0.01 #s
t_max = 500.0 #s
m = 1 #massa unitaria
t = np.arange(t_0, t_max, h)

E_0 = 0.5*m*(L*omega_in)**2 + m*g*L*(1 - np.cos(theta_0))

def energia(theta, omega, L, m, g):
    return 0.5*m*(L*omega)**2 + m*L*g*(1 - np.cos(theta))

theta_verlet, omega_verlet, a_verlet = velocity_verlet(t, h, theta_0, omega_in, a_0, L, g)
theta_RK4, omega_RK4 = RK4(t, h, theta_0, omega_in, L, g)

energia_verlet = energia(theta_verlet, omega_verlet, L, m, g)
energia_RK4    = energia(theta_RK4, omega_RK4, L, m, g)

delta_E_verlet = E_0 - energia_verlet
delta_E_RK4    = E_0 - energia_RK4

U_V = (delta_E_verlet/E_0)*100
U_RK4 = (delta_E_RK4/E_0)*100

plt.plot(t, U_V)
plt.xlabel("t (s)")
plt.ylabel(r"$\Delta E/E_0$")
plt.grid(True)
plt.title('Variazione Verlet')
plt.show()

plt.plot(t, U_RK4)
plt.xlabel("t (s)")
plt.ylabel(r"$\Delta E/E_0$")
plt.title('Variazione RK4')
plt.grid(True)
plt.show()
```

```python
#Esercizio 3

plt.plot(theta_verlet, omega_verlet, 'r-')
plt.xlabel(r"$\theta (t)$ (rad)")
plt.ylabel(r"$\omega (t)$ (rad/s)")
plt.grid(True)
plt.title(r'$\omega$ vs $\theta$ Verlet')
plt.show()

plt.plot(theta_RK4, omega_RK4, 'b-')
plt.xlabel(r"$\theta (t)$ (rad)")
plt.ylabel(r"$\omega (t)$ (rad/s)")
plt.grid(True)
plt.title(r'$\omega$ vs $\theta$ RK4')
plt.show()

#ordine di convergenza

h_values = [0.1, 0.05, 0.02, 0.01, 0.005, 0.002, 0.001]
errore_verlet = []
errore_RK4    = []

h_c = 0.0001
t_finale = 10.0
N_c   = round(t_finale / h_c)
t_c   = np.linspace(0, t_finale, N_c + 1)
theta_0 = 0.5
omega_in = 0
L = 1
g = 9.81
a_0 = -(g/L)*np.sin(theta_0)

theta_RK4_c = RK4(t_c, h_c, theta_0, omega_in, L, g)[0]
theta_RK4_c_last = theta_RK4_c[-1]

for h in h_values:
    N = round(t_finale / h)
    t = np.linspace(0, t_finale, N + 1)

    theta_verlet = velocity_verlet(t, h, theta_0, omega_in, a_0, L, g)[0]
    theta_RK4 = RK4(t, h, theta_0, omega_in, L, g)[0]

    diff_verlet = theta_verlet[-1] - theta_RK4_c_last
    diff_RK4    = theta_RK4[-1] - theta_RK4_c_last

    errore_verlet.append(np.abs(diff_verlet))
    errore_RK4.append(np.abs(diff_RK4))

plt.plot(h_values, errore_verlet, 'r-o', label='Errore Verlet')
plt.plot(h_values, errore_RK4, 'b-o', label='Errore RK4')
plt.legend()
plt.grid(True)
plt.xscale('log')
plt.yscale('log')
plt.xlabel(r"$h$")
plt.ylabel(r"$\varepsilon$")
plt.show()

# Stima ordine di convergenza
log_h = np.log(h_values)
ordine_verlet = np.polyfit(log_h, np.log(errore_verlet), 1)[0]
```

```python
ordine_RK4     = np.polyfit(log_h, np.log(errore_RK4),     1)[0]
print(f"Ordine Verlet: {ordine_verlet:.2f}")  # atteso 2, esce 1.99
print(f"Ordine RK4:    {ordine_RK4:.2f}") # atteso 4, esce 4.04
```